# Lab 7: Indexes and Tuning

In this lab you will gain hands-on experience with database indexes and query tuning. You will use the EXPLAIN and EXPLAIN ANALYZE commands to understand how the PostgreSQL query planner executes queries, measure the performance impact of indexes, and learn to design indexes for specific workloads.

The IMDB database you will use contains **over 26 million rows** across six tables — large enough that the difference between a sequential scan and an index scan is clearly visible in timing.

**Schema:**

```
Actor(id, fname, lname, gender)          -- 3,442,863 rows
Movie(id, name, year)                    -- 2,383,721 rows
Directors(id, fname, lname)              -- 583,174 rows
Casts(pid, mid, role)                    -- 16,927,847 rows
Movie_Directors(did, mid)                -- 3,250,604 rows
Genre(mid, genre)                        -- 956,674 rows
```

**Objectives:**

- Understand which indexes PostgreSQL creates automatically.
- Measure query execution time with \timing on.
- Use EXPLAIN and EXPLAIN ANALYZE to read and interpret query plans.
- Identify performance bottlenecks in multi-table queries.
- Design single-attribute and composite indexes for specific access patterns.
- Recognize when an index improves performance — and when it does not.

***Note on measuring query time***

*Enable timing in psql with: \timing on*

*The first execution of a query is often slower because PostgreSQL has not yet loaded the relevant data pages into its buffer cache. When benchmarking, discard the first run and average the rest.*

# Part 1: Exploring indexes

This part is exploratory. You do not submit any files for Part 1, but you should work through all the steps — they prepare you for the graded questions in Part 2.

## 1.1 Listing existing indexes

Connect to your IMDB database and run:

```
\di
```

PostgreSQL automatically creates indexes for primary key and unique constraints. For each index listed, note the table it covers and which column(s) are indexed. This is the baseline you are starting from.

## 1.2 Timing a query without an index

Enable timing:

```
\timing on
```

and run the following query several times:

```
SELECT *
FROM Actor
WHERE lname = 'Bacon';
```

Run this query five times and record each time. The Actor table has 3.4 million rows, so the sequential scan required to find all actors named "Bacon" should take a noticeable amount of time.

## 1.3 EXPLAIN and EXPLAIN ANALYZE

Use EXPLAIN to see the query plan without executing the query:

```
EXPLAIN
SELECT *
FROM Actor
WHERE lname = 'Bacon';
```

You should see a Seq Scan (sequential scan) — PostgreSQL reads every one of the 3.4 million rows to find the matching records. Note the estimated row count.

Now use EXPLAIN ANALYZE to execute the query and compare estimates against actual statistics:

```
EXPLAIN ANALYZE
SELECT *
FROM Actor
WHERE lname = 'Bacon';
```

Look at the **actual time** and **actual rows** values. How close are the planner's estimates to reality?

## 1.4 Creating a single-attribute index

Create an index on actor last name:

```
CREATE INDEX actorlname ON Actor(lname);
```

Creating the index over 3.4 million rows may take a little bit, depending on your machine. Once it finishes, run the Bacon query again several times and compare the new times. Re-examine the query plan:

```
EXPLAIN ANALYZE
SELECT *
FROM Actor
WHERE lname = 'Bacon';
```

You should now see an Index Scan (or Bitmap Index Scan) instead of a Seq Scan.

Before moving on, drop this index so you start Part 2 with a clean slate:

```
DROP INDEX actorlname;
```

## 1.5 Column order in composite indexes

Create a composite index with first name listed first:

```
CREATE INDEX actor_fname_lname ON Actor(fname, lname);
```

Run EXPLAIN on each of the following three queries and observe which plan is used for each:

```
EXPLAIN SELECT * FROM Actor WHERE lname = 'Bacon';
EXPLAIN SELECT * FROM Actor WHERE fname = 'Kevin' AND lname = 'Bacon';
EXPLAIN SELECT * FROM Actor WHERE fname = 'Kevin';
```

**Key concept:** A composite index on (A, B) supports lookups on A alone or on A and B together. It ***cannot*** efficiently support a lookup on B alone — the leftmost column must be specified in the predicate. This makes column ordering a critical design decision for composite indexes.

Before moving on, drop this index:

```
DROP INDEX actor_fname_lname;
```

## 1.6 Selectivity: when indexes help and when they do not

Run the following query and look at the plan:

```
EXPLAIN SELECT * FROM Actor WHERE gender = 'M';
```

**Selectivity** measures how much an index can narrow down the result. A column with very few distinct values (like the gender column) has *low selectivity*. When a predicate on a low-selectivity column would return a large fraction of the table anyway, PostgreSQL often prefers a sequential scan over an index scan, even if an index exists. Create an index and verify:

```
CREATE INDEX actor_gender ON Actor(gender);
EXPLAIN SELECT * FROM Actor WHERE gender = 'M';
EXPLAIN SELECT * FROM Actor WHERE gender = 'F';
```

Does PostgreSQL use the index? Why or why not? When you are done exploring, drop this index.

## Part 2: Creating indexes

For each question below, analyze the given query to identify why it is slow, then write a `CREATE INDEX` statement that addresses the bottleneck. Each question is submitted as a separate file. Your .sql file should contain exactly one CREATE INDEX statement and nothing else (no DROP INDEX, no SELECT, no other DDL).

**Grading:** The autograder checks that an appropriate index exists on the correct table and column(s). You may choose any valid index name.

### Question 1 (Q1.sql)

The following query looks up a movie by title. It currently performs a full sequential scan over the Movie table (2.4 million rows).

```
SELECT id, name, year
FROM Movie
WHERE name = 'Casablanca';
```

Use EXPLAIN to confirm the current plan. Identify which table and column are the bottleneck, and create an appropriate index. Verify with EXPLAIN that the plan changes.

**Question 1**

Submit Q1.sql containing exactly one CREATE INDEX statement that speeds up the query above.

### Question 2 (Q2.sql)

The following query retrieves the cast list for a movie, joining three tables.

```
SELECT A.fname, A.lname, C.role
FROM Actor A, Casts C, Movie M
WHERE M.name = 'Titanic' AND A.id = C.pid AND C.mid = M.id;
```

Assume that your index from Q1 is already in place, but it is not required for the query plan to use this index. Run EXPLAIN on the full query and examine the plan carefully; look at *each* join and each scan. Which step is still performing a sequential scan despite the Q1 index? Identify the remaining bottleneck and create an index to fix it.

*Hint: Think about what the join condition requires. When PostgreSQL joins two tables on a column, it benefits greatly from an index on the join column in the larger table.*

**Question 2**

Submit Q2.sql containing exactly one CREATE INDEX statement that addresses the remaining bottleneck in the query above (assuming Q1's index exists).

## Question 3 (Q3.sql)

The following query retrieves all actors who appeared in Action movies.

```
SELECT DISTINCT A.fname, A.lname
FROM Actor A, Casts C, Genre G
WHERE G.genre = 'Action' AND A.id = C.pid
          AND C.mid = G.mid;
```

Assume that your indexes from Q1 and Q2 are already in place, but it is not required for the query plan to use these indexes. Run EXPLAIN and identify the remaining bottleneck. Think carefully: is there an index design that is not only correct but especially efficient for this query because of how PostgreSQL can satisfy both the filter predicate and the join condition from a single index lookup?

*Grading note: Q3 has partial and full credit.*

*A correct but basic index earns partial credit. The best index design (one that also eliminates a table lookup for the filtered rows) earns full credit.*

**Question 3**

Submit Q3.sql containing exactly one CREATE INDEX statement that best speeds up the query above (assuming Q1's and Q2's indexes exist).

# Part 3: Tuning for a workload

In this part you will tune a different database for a given query workload. Unlike Parts 1 and 2, here you must analyze multiple queries at once and decide on a set of indexes that improves the whole workload as much as possible.

## Schema

The UMassMMC database represents a small medical center:

```
Patient(pid, fname, lname, age, street, city, zipcode)   -- PK: pid
Disease(pid, disease)          -- PK: (pid, disease); FK: pid → Patient
Doctor(did, fname, lname, specialty)                     -- PK: did
Sees(pid, did)                 -- PK: (pid, did);
                               -- FKs: pid → Patient, did → Doctor
Product(eid, description)      -- PK: eid
Stock(eid, quantity)           -- PK: eid; FK: eid → Product
Supplier(sid, name, street, city, zipcode)               -- PK: sid
Supplies(eid, sid)             -- Unique: (eid, sid)
```

The database is populated at moderate scale (tens of thousands of rows). PostgreSQL automatically creates indexes for all primary key and unique constraints.

## Query workload

The following six queries represent the most common access patterns in the UMassMMC application. The ? parameters vary randomly across all distinct values each time a query runs, and each query runs many thousands of times.

**W1 — look up doctors by specialty:**

```
SELECT fname, lname
FROM Doctor
WHERE specialty = ?;
```

**W2 — find all doctors for a given patient (identified by name):**

```
SELECT D.fname, D.lname
FROM Doctor D, Sees S, Patient P
WHERE D.did = S.did
```

```
  AND S.pid = P.pid
  AND P.fname = ?
  AND P.lname = ?;
```

**W3 — find all patients for a given doctor (identified by name):**

```
SELECT P.fname, P.lname
FROM Doctor D, Sees S, Patient P
WHERE D.did = S.did
  AND S.pid = P.pid
  AND D.fname = ?
  AND D.lname = ?;
```

**W4 — count how many patients have a given disease:**

```
SELECT disease, count(*)
FROM Disease
WHERE disease = ?
GROUP BY disease;
```

**W5 — count patients within a narrow age range:**

```
SELECT count(*)
FROM Patient
WHERE age > ? AND age < ?;
```

**W6 — count patients in a given zip code:**

```
SELECT count(*)
FROM Patient
WHERE zipcode = ?;
```

> *Your goal is to design a set of indexes that speeds up this workload as a whole.*
>
> *Analyze each query: what table does it filter or join on, and on which column(s)?*
>
> *Think about both single-attribute and composite indexes.*

**Practice data and simulate workload:** While it is not strictly necessary to solve this part, you may find it helpful to test your indexes on the actual database, using the EXPLAIN command. Please follow the instructions in this document to set up the UMassMMC database and to simulate this workload.

### Question 4 (Q4.sql)

Analyze queries W1–W6 and submit `Q4.sql` containing the CREATE INDEX statements you judge most beneficial. There is no fixed required number; create the indexes you believe best serve the workload. You may choose any valid index names.

**Grading:** The autograder checks for each expected index independently using the catalog. Each correct index earns partial credit regardless of whether you submitted the others.

> **Question 4**
>
> Submit Q4.sql with all the CREATE INDEX statements you judge necessary to speed up queries W1–W6. Each correct index earns points independently.

## Part 4: Index selectivity

Not every column benefits from an index. This part asks you to think critically about when indexes help and when they do not. You do not submit any files for Part 4.

### 4.1 A low-selectivity column

**Selectivity** measures how much an index narrows down the result set. Run the following to see the value distribution of one column:

```
SELECT gender, COUNT(*) FROM Actor GROUP BY gender;
```

Create an index on this column, then check the query plan for a predicate on it. Does PostgreSQL use the index? Experiment with a different predicate value and see if the behavior changes.

## 4.2 The cost of indexes on write-heavy tables

Every index speeds up reads but **slows down writes**: each INSERT, UPDATE, or DELETE must update all indexes on the affected table. For the Casts table with 16.9 million rows, bulk loads and frequent updates carry significant overhead. As a rule of thumb: index the columns that appear in WHERE clauses and join conditions of your most frequent queries. Avoid indexing columns that are rarely queried or that have very low cardinality.

## 4.3 Range queries

Compare the query plans for an equality predicate versus a wide range predicate on the same column:

```
EXPLAIN SELECT * FROM Movie WHERE year = 1995;
EXPLAIN SELECT * FROM Movie WHERE year > 1990;
EXPLAIN SELECT * FROM Movie WHERE year > 1900;
```

First, create an index on `year` and then run the above EXPLAIN queries. For the equality predicate the planner should use the index. For the range predicate that would return a large fraction of the table, the planner may switch to a sequential scan: it decided that walking a large portion of the index and heap costs more than a single sequential pass. Drop the index when done.